

Hadoop Installation & MapReduce Execution Guide

Step-by-step procedure for Pseudo-Distributed Apache Hadoop setup and MapReduce program execution

Part 1: Installing and Configuring Apache Hadoop

Step 1: Install OpenJDK 8

Update apt package repository and install Java 8 JDK:

```
sudo apt update
sudo apt install openjdk-8-jdk -y
```

Verify installation and find your Java installation path:

```
java -version
readlink -f /usr/bin/java | sed "s:bin/java::"
```

Note: Typical path output is /usr/lib/jvm/java-8-openjdk-amd64/

Step 2: Configure SSH (Passphraseless Access)

Hadoop requires passwordless SSH access to manage daemon nodes locally:

```
sudo apt install openssh-server openssh-client -y
ssh-keygen -t rsa -P "" -f ~/.ssh/id_rsa
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 0600 ~/.ssh/authorized_keys
ssh localhost
```

(Type exit to exit the SSH session once verified).

Step 3: Download and Extract Apache Hadoop

Download Hadoop 3.3.6 and extract it to your home directory:

```
wget https://dlcdn.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
tar -xvzf hadoop-3.3.6.tar.gz
mv hadoop-3.3.6 ~/hadoop
```

Step 4: Configure Environment Variables

Edit ~/.bashrc and append the following configurations:

```
nano ~/.bashrc
```

Add at the bottom of the file:

```
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
export HADOOP_HOME=$HOME/hadoop
export HADOOP_INSTALL=$HADOOP_HOME
export HADOOP_MAPRED_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_HDFS_HOME=$HADOOP_HOME
export YARN_HOME=$HADOOP_HOME
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
```

Apply the environment updates:

```
source ~/.bashrc
```

Step 5: Configure Hadoop XML Configuration Files

Navigate to \$HADOOP_HOME/etc/hadoop and configure the following files:

1. hadoop-env.sh

```
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
```

2. core-site.xml

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
```

3. hdfs-site.xml

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
</configuration>
```

4. mapred-site.xml

```
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
```

5. yarn-site.xml

```
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
</configuration>
```

Step 6: Format NameNode & Start Services

Format HDFS filesystem and start Hadoop daemons:

```
hdfs namenode -format
start-dfs.sh
start-yarn.sh
```

Verify running daemons:

```
jps
```

Active daemons should include: NameNode, DataNode, SecondaryNameNode, ResourceManager, NodeManager.

Part 2: MapReduce WordCount Program Execution

Step 1: Create Directory and Source File

```
mkdir ~/wordcount
cd ~/wordcount
nano WordCount.java
```

Java Source Code (WordCount.java):

```
import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    public static class TokenizerMapper
        extends Mapper<Object, Text, Text, IntWritable>{

        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(Object key, Text value, Context context
            ) throws IOException, InterruptedException {
            StringTokenizer itr = new StringTokenizer(value.toString());
            while (itr.hasMoreTokens()) {
                word.set(itr.nextToken());
                context.write(word, one);
            }
        }
    }

    public static class IntSumReducer
        extends Reducer<Text, IntWritable, Text, IntWritable> {
        private IntWritable result = new IntWritable();

        public void reduce(Text key, Iterable<IntWritable> values,
            Context context
            ) throws IOException, InterruptedException {
            int sum = 0;
            for (IntWritable val : values) {
                sum += val.get();
            }
            result.set(sum);
            context.write(key, result);
        }
    }

    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "word count");
        job.setJarByClass(WordCount.class);
        job.setMapperClass(TokenizerMapper.class);
```

```

    job.setCombinerClass(IntSumReducer.class);
    job.setReducerClass(IntSumReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

Step 2: Compile Code and Package Executable JAR

Compile Java class with Hadoop classpath dependencies and build JAR file:

```

javac -classpath `hadoop classpath` -d . WordCount.java
jar -cvf wordcount.jar *.class

```

Step 3: Prepare Input Data & Upload to HDFS

Create sample text file and put it into HDFS input path:

```

echo "hello hadoop hadoop mapreduce" > input.txt
hdfs dfs -mkdir -p /home/hadoop/input
hdfs dfs -put input.txt /home/hadoop/input/

```

Step 4: Execute MapReduce Job & Inspect Output

Run the MapReduce program via Hadoop jar command:

```

hadoop jar wordcount.jar WordCount /home/hadoop/input /home/hadoop/output

```

Check the execution output from HDFS:

```

hdfs dfs -cat /home/hadoop/output/part-r-00000

```

Expected Output:

```

hadoop 2
hello 1
mapreduce 1

```

Troubleshooting & Common Fixes

Issue: DataNode not running or Connection Refused:

```
stop-all.sh  
rm -rf /tmp/hadoop-$(whoami)/dfs  
hdfs namenode -format  
start-dfs.sh  
start-yarn.sh
```

Issue: Output directory already exists error:

```
hdfs dfs -rm -r /home/hadoop/output
```